

# 阅读文献笔记

[以防笔记篇幅过长，所需的前置知识已放在“前置知识.word”]

## 一、vLLM / PagedAttention (SOSP 2023)

- **解决的核心痛点**：大模型推理时，KV Cache 非常吃显存。以前的做法很“笨”，直接给每个请求按最大可能长度预分配一整块连续显存，导致严重的内部碎片和外部碎片，显存利用率只有 20%-38%。
- **核心创新点 (PagedAttention)**：作者借鉴了操作系统的“虚拟内存分页”思想。把 KV Cache 切成固定大小的 Block，存在非连续的物理显存里。配合 Block 表进行映射，**按需分配**。
- **关键机制**：引入了“写时复制”(Copy-on-Write)，在并行采样或 Beam Search 时，不同分支可以共享大部分物理 Block，只有需要修改时才复制一份。
- **启发**：这让我彻底理解了 SGLang 中 Page-sized KV Cache (分页管理) 的底层原理。它把显存利用率拉到 90% 以上，是工业界推理系统的标配。

## 二、SGLang / RadixAttention (NeurIPS 2024)

- **解决的核心痛点**：在多轮对话、Few-shot 或 Agent 任务中，很多请求的前缀（如系统提示词）是一模一样的。传统框架无法自动识别和复用这些复杂的前缀，导致大量重复计算和显存浪费。
- **核心创新点 (RadixAttention)**：用一棵“**基数树 (Radix Tree)**”来管理 KV Cache。新请求进来，自动匹配最长前缀，命中就直接复用，跳过 Prefill 计算。
- **配套优化**：引入了“**缓存感知调度**”（优先处理前缀匹配度高的请求）来提升命中率；同时用“**压缩有限状态机**”加速 JSON 等结构化输出。
- **启发**：这正是作业中 RadixCache 的来源。它让多轮对话场景下的缓存命中率能达到 75% - 90%，显著降低了首字延迟 (TTFT)。

### 三、两者关系与总结

- **联系：**两者都采用了 Page/Block 粒度的 KV Cache 管理机制，也都支持前缀共享。
- **差异：**vLLM 的 PagedAttention 属于底层的内存管理层创新（打好地基）；SGLang 的 RadixAttention 是在其之上加入了前缀语义层的创新（盖起精装房），能自动匹配任意长度的树状前缀。
- **个人反思：**通过本次挑战，我从理论层面理解了这些优化，但在实操中（本地 AMD 显卡、云端缺 C++ 编译器、TorchAudio 版本冲突）深刻体会到了环境依赖和系统架构对于推理框架落地的决定性影响。